FASTAPI 식당 예약 · 6인 팀 · 2026-10-26 ~ 11-06

식당 예약 프로젝트 흐름도

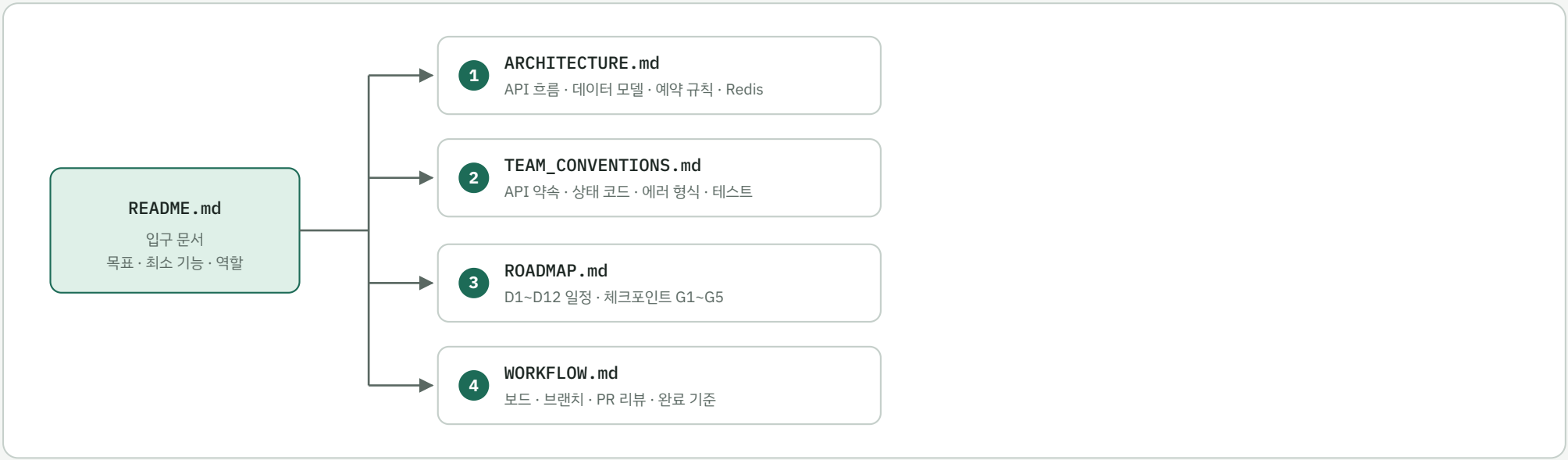
팀 문서에 흩어진 백엔드 API 흐름을 그림으로 모았습니다. 요청 하나가 서버 안에서 어떻게 처리되는지부터 보고, 일정과 작업 방식은 마지막에 봅니다. 세부 규칙은 각 그림 아래에 적은 원본 문서에서 확인하세요.

원본 문서: 루트 `README.md`와 `docs/project/`의 md 4개

초점: 백엔드 API (FastAPI · PostgreSQL · Redis). UI는 이 API를 호출하는 쪽으로만 다룹니다.

문서를 읽는 순서

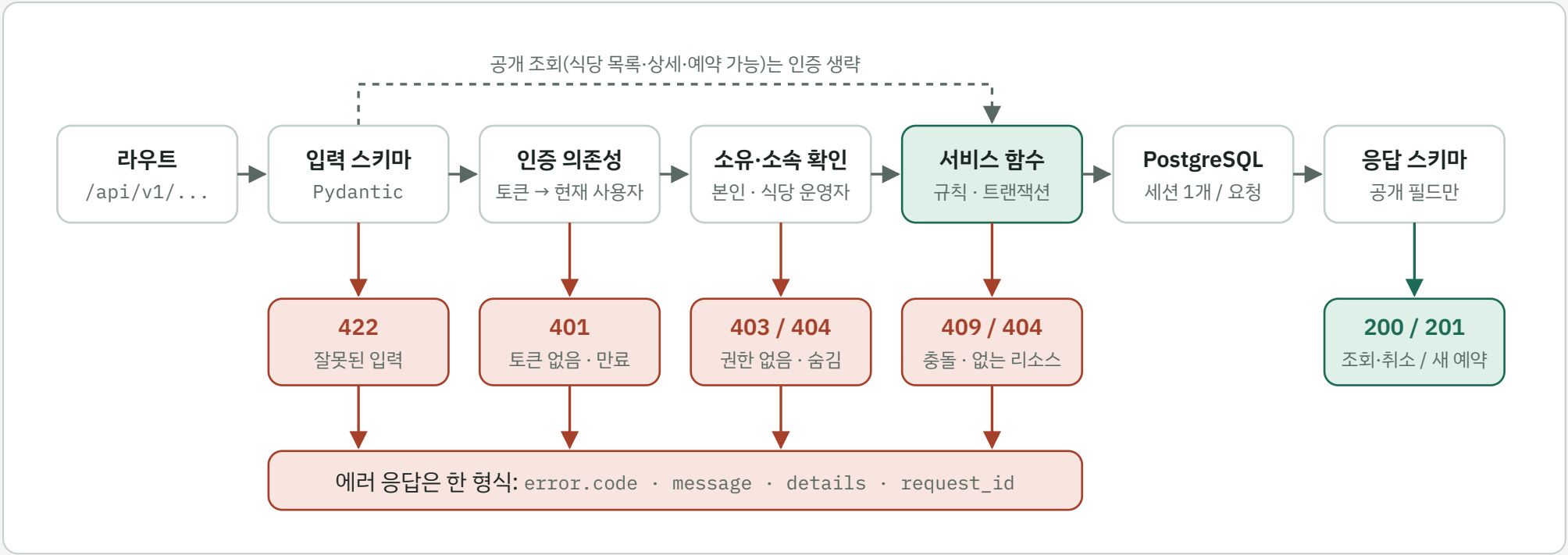
`README.md · 팀 문서 표`



백엔드 API를 먼저 이해하려면 1→2 순서가 핵심입니다. ARCHITECTURE가 "어떤 API가 어떤 규칙으로 동작하는지", CONVENTIONS가 "응답과 에러를 어떤 모양으로 돌려주는지"를 정합니다. 3~4번은 일정과 협업 방식입니다.

요청 하나가 서버 안에서 처리되는 흐름

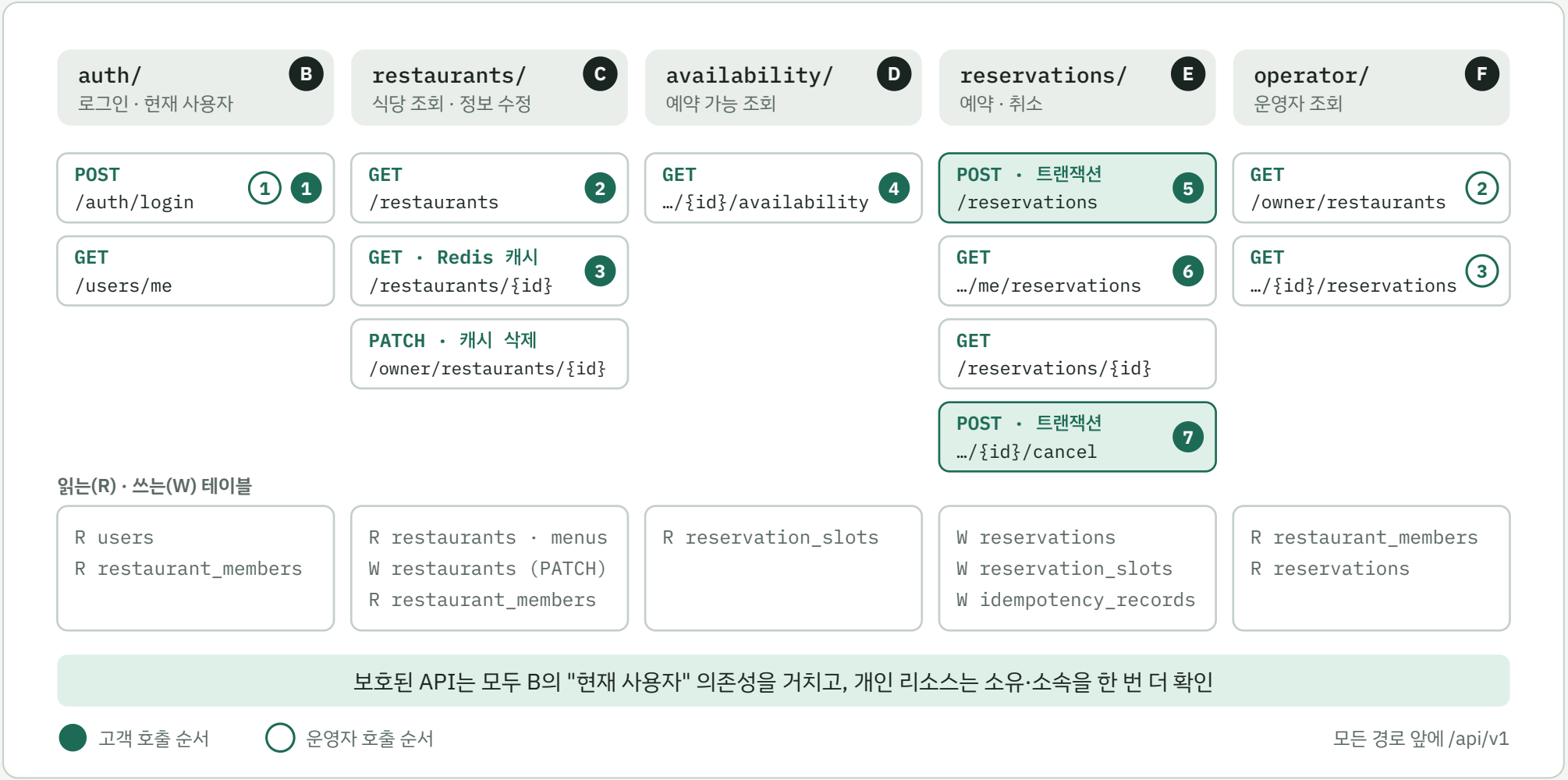
ARCHITECTURE.md §1, §2 · TEAM_CONVENTIONS.md §3, §4



모든 기능이 같은 통로를 지나고, 실패하는 위치에 따라 상태 코드가 정해집니다. 인증 의존성은 B가 만들고 모두가 씁니다. 로그인만으로는 부족해서, 개인 리소스는 소유·소속을 한 번 더 확인합니다. 관련 없는 사람에게는 403 대신 404로 존재 자체를 숨길 수 있습니다. 커밋은 서비스 함수에서만 하고, 그 안에서 부르는 쿼리 헬퍼는 커밋하지 않습니다. UI는 에러 메시지 문구가 아니라 `error.code`를 보고 분기합니다.

API 지도: 누가 어떤 엔드포인트와 테이블을 맡나

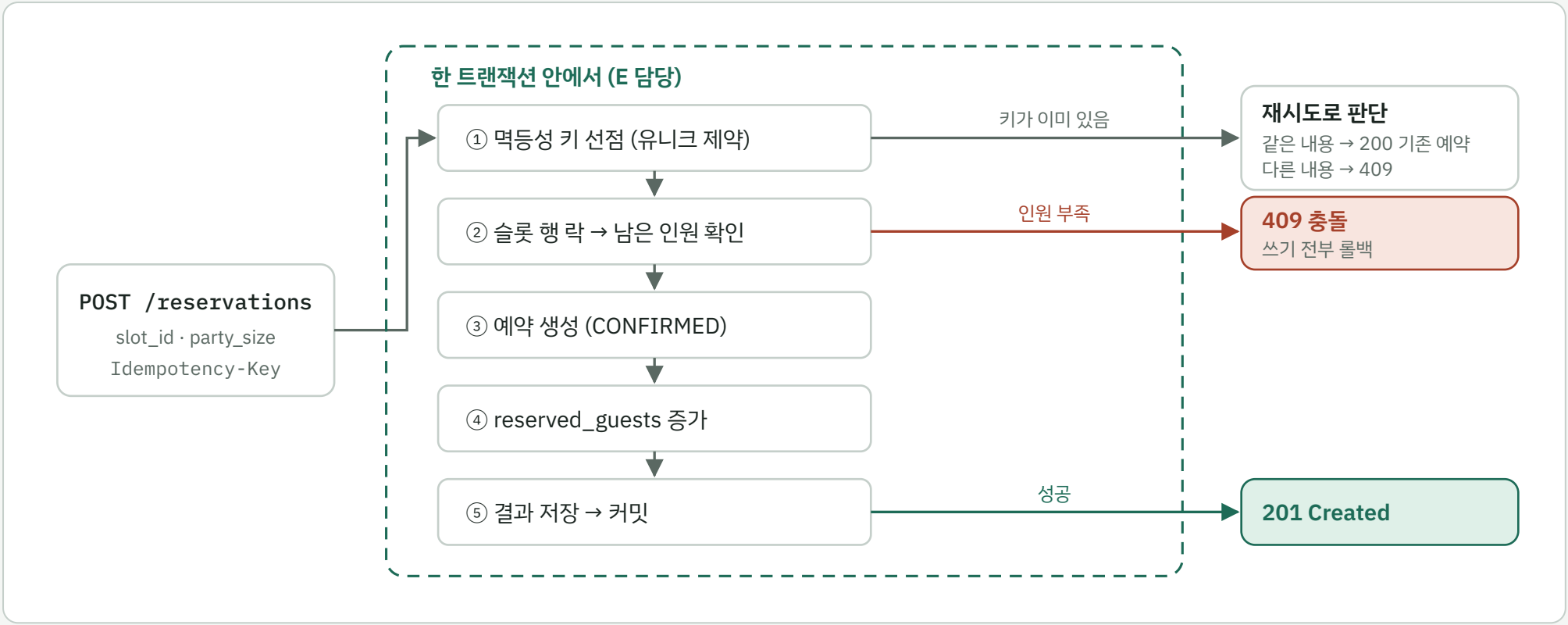
ARCHITECTURE.md §2, §3, §5



엔드포인트 12개가 기능 모듈 5개에 나뉘어 있습니다. 시연 때 고객은 채운 원 1→7 순서로, 운영자는 빈 원 1→3 순서로 API를 호출합니다. 상태를 바꾸는 쓰기는 E의 두 엔드포인트(초록 칸)에 몰려 있어 트랜잭션도 여기서만 엽니다. C의 수정 API는 캐시 무효화를 연습하려고 넣은 쓰기입니다. 예약 가능 조회 결과는 그 순간의 스냅샷이라, 실제 예약할 때 E가 인원을 다시 확인합니다.

예약 생성: 한 트랜잭션 안의 다섯 단계

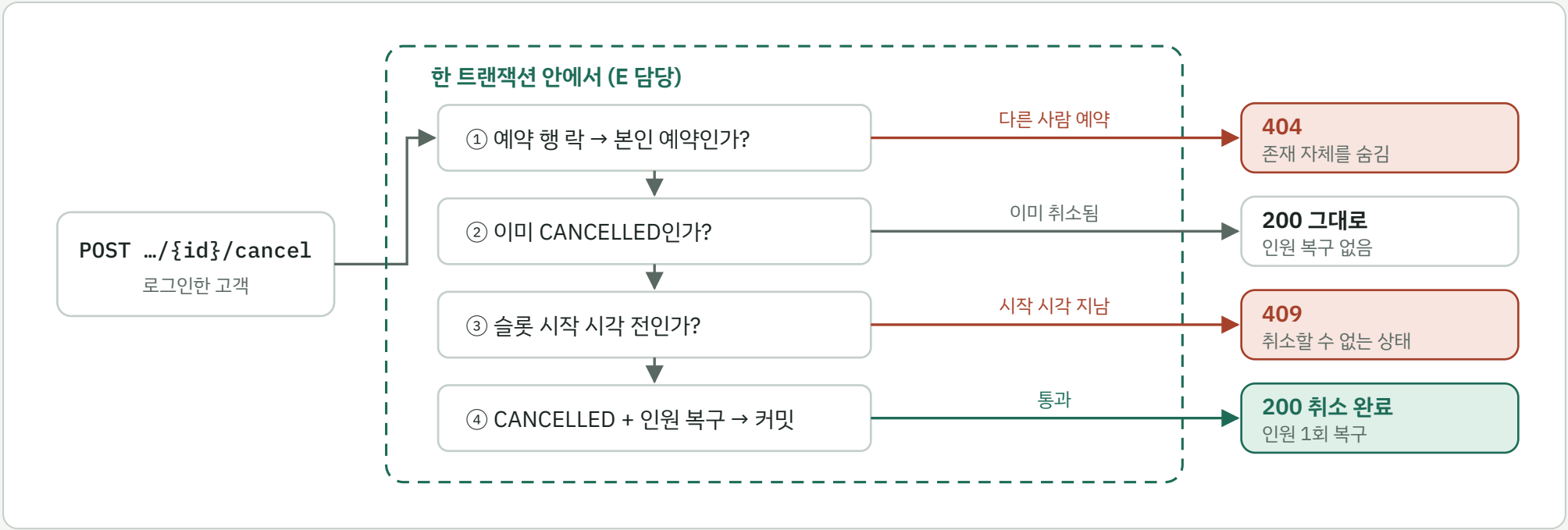
ARCHITECTURE.md §4, §6



다섯 단계가 한 트랜잭션이라서, 동시에 들어온 두 요청 중 하나만 마지막 자리를 가져갑니다. 두 요청이 같은 잔여 인원을 볼 수는 있지만, ②의 행 락 때문에 차례로 확인하게 됩니다. 인원은 1~8명이고, 슬롯 하나에 전원이 들어가야 합니다. 락 순서는 모든 쓰기 경로에서 같게 하고, 락 대기 시간에 제한을 둡니다. 수용 인원 판단에 Redis는 쓰지 않습니다.

예약 취소: 여러 번 눌러도 인원은 한 번만 복구

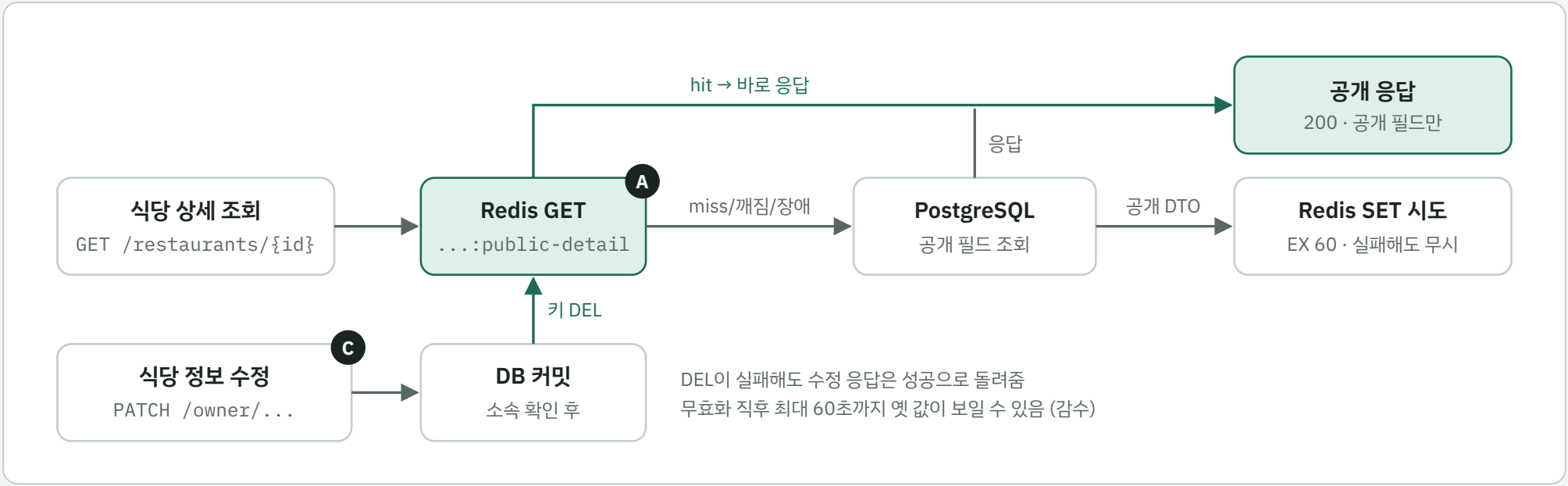
ARCHITECTURE.md §4, §6



②에서 이미 취소된 예약은 아무것도 바꾸지 않고 성공을 돌려줍니다. 그래서 취소 버튼을 여러 번 눌러도 결과가 같고, `reserved_guests`는 한 번만 줄어듭니다. 예약 상태는 `CONFIRMED`와 `CANCELLED` 두 가지뿐입니다. 이 동작은 독립된 PostgreSQL 세션으로 요청을 일부러 겹치게 보내서 테스트합니다.

식당 상세 캐시: 백엔드 안에서만 쓰는 Redis

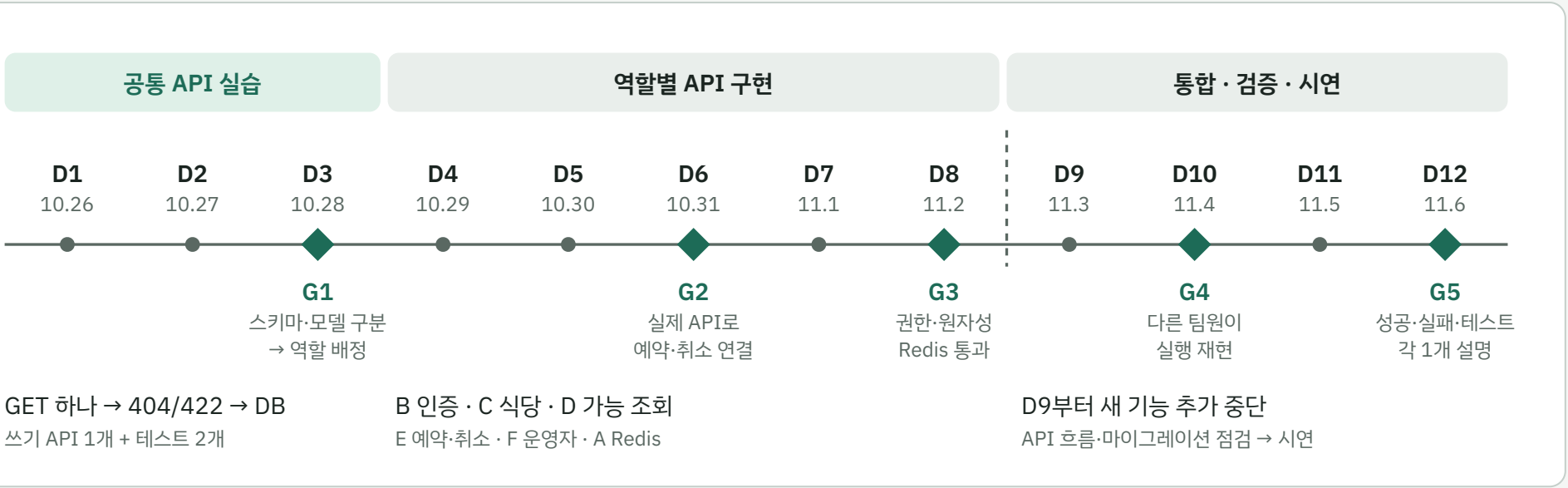
ARCHITECTURE.md §7 · TEAM_CONVENTIONS.md §6



Redis가 죽어도 조회는 DB로 계속 동작합니다. DB가 죽으면 그대로 장애입니다. 키는 `reservation-demo:{environment}:v1:restaurant:{id}:public-detail`이고, hit이 나도 TTL을 늘리지 않습니다. 로그인 정보, 권한, 개인 예약, 남은 수용 인원은 캐싱하지 않습니다. 어댑터·키·TTL은 A가, 응답 형태와 무효화 호출 위치는 C가 맡습니다. Redis가 준비되기 전에는 아무것도 안 하는 no-op 어댑터로 개발합니다.

12일 동안 API가 만들어지는 순서

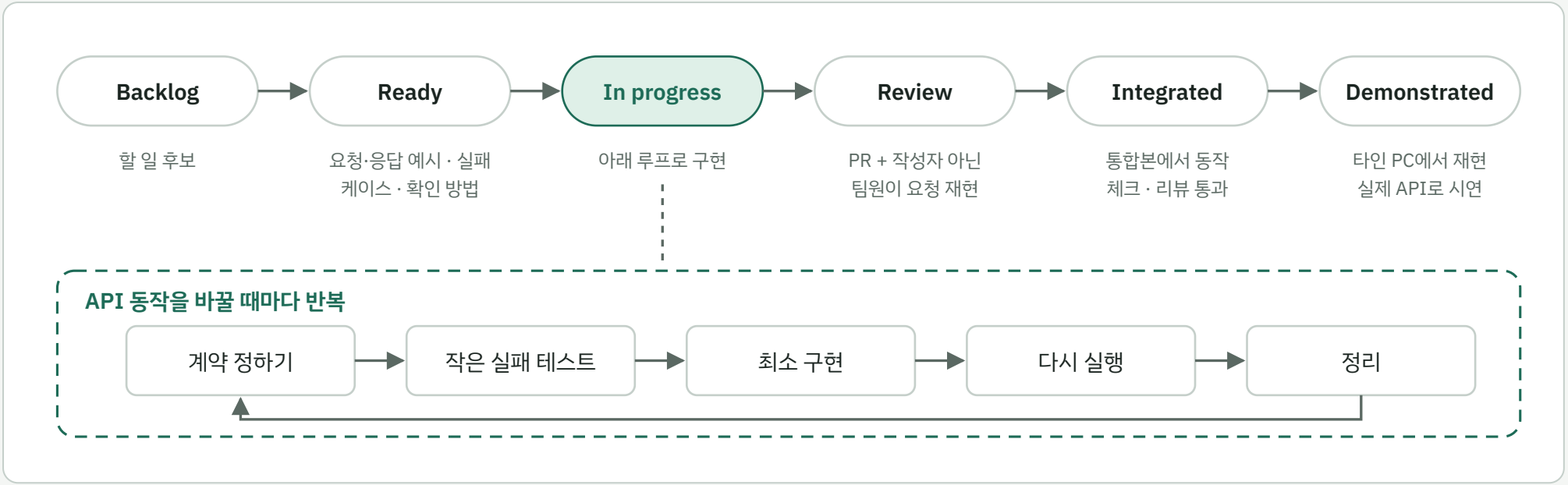
ROADMAP.md · 일자별 계획, 체크포인트



마름모가 체크포인트(G1~G5)입니다. D1~D3에는 모두가 같은 작은 API(메모리 리스트 → ID 검증 → 없으면 404 → DB 조회)를 만들고, 그중 하나만 기준 예제로 통합합니다. G2는 프론트 목업 없이 여러 사람이 만든 실제 API를 이어서 통과해야 합니다. 점선(D9) 이후로는 새 기능 없이 버그 수정과 재현만 합니다.

API 작업 하나가 완료되기까지

WORKFLOW.md §3, §4, §5, §7



머지됐다고 끝이 아니고, 다른 팀원 컴퓨터에서 재현되어야 Demonstrated입니다. 계약은 성공 요청·응답 예시 1개와 실패 예시 1개입니다. 화면에 연결하기 전에 API 문서 화면에서 요청이 되는 걸 먼저 보여 줍니다. 개인 리소스는 관련 없는 사용자로 호출해 거부되는지, 예약·취소는 실제 DB로 동시 요청과 재시도를 확인합니다. CI 통과만으로는 머지하지 않습니다.